ml

Tanvir

(data + learning algorithm) $\rightarrow$ function

1 Supervised learning

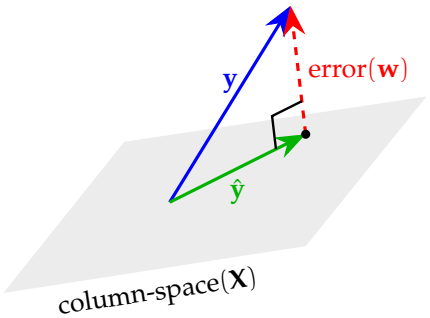
1.1 Regression

model, parameters, cost function, objective

1.1.1 Linear (in w) Regression

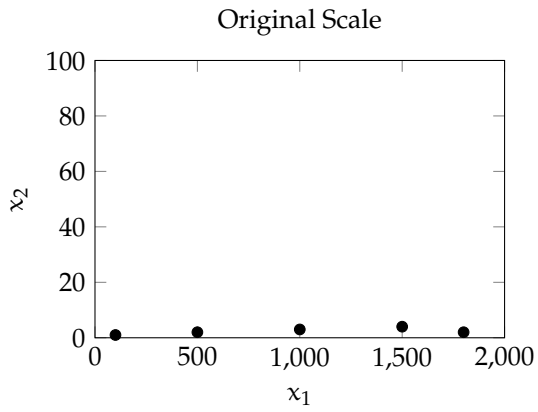
$\mathbf{X}^{m\times n}$ has m examples, each having n features. Usually $m \gg n$. $\mathbf{y}^{m\times 1}$ are the corresponding outputs.

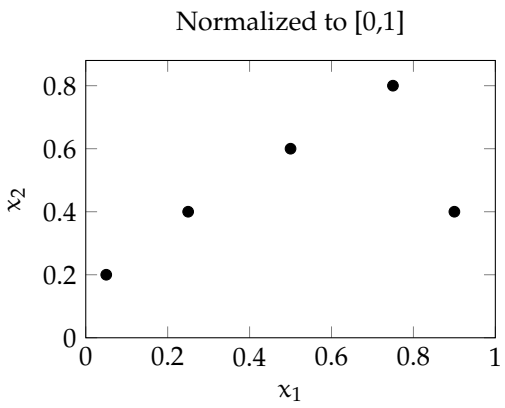
$$
\begin{aligned}
\mathbf{X}\mathbf{w} &= \hat{\mathbf{y}} \\
\text{error}(\mathbf{w}) &= \mathbf{y} - \hat{\mathbf{y}} \\
\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w}) &= \frac{1}{m}\|\text{error}(\mathbf{w})\|^2 = \frac{1}{m}(\mathbf{X}\mathbf{w}-\mathbf{y})^T(\mathbf{X}\mathbf{w}-\mathbf{y}) \\
\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} &= \frac{2}{m}\mathbf{X}^T(\mathbf{X}\mathbf{w}-\mathbf{y}) \\
\texttt{gradient descent:}\quad \mathbf{w} &= \mathbf{w} - \alpha\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}
\end{aligned}
$$

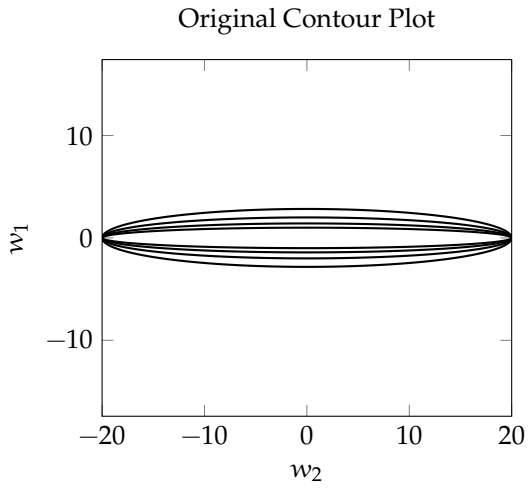


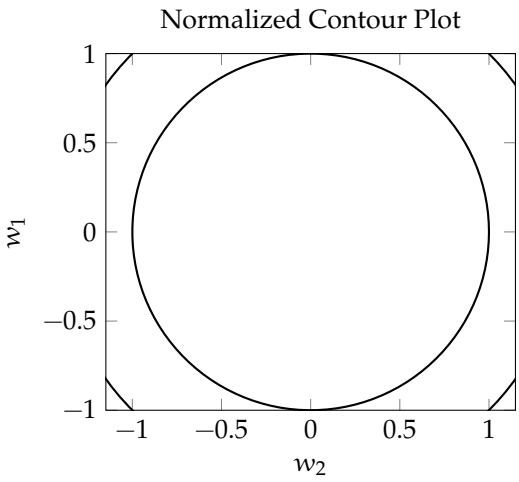
1.2 Normalization

Changes scale keeping the shape of distribution same. Gradient descent now works, with more ease, in the world of concentric circular contours.



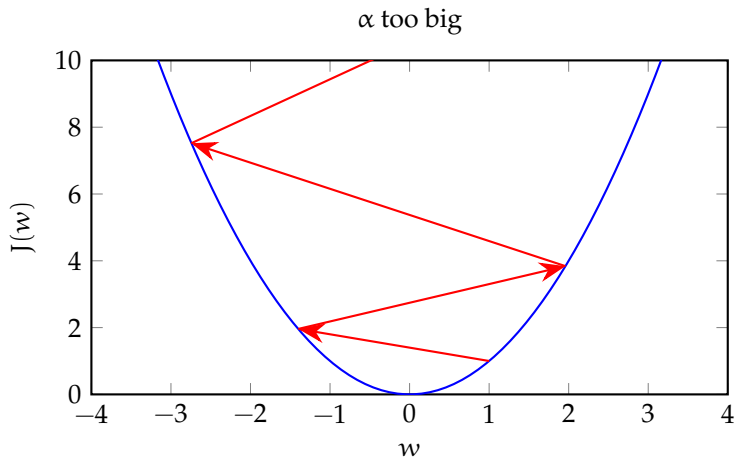






1.3 Learning rate, α

A good value is somewhere between too big (divergence) and too small (slow convergence).



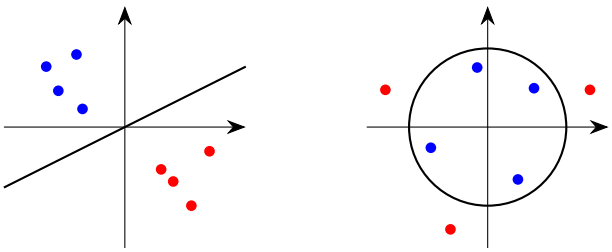
1.4 Classification

1.4.1 Logistic Regression

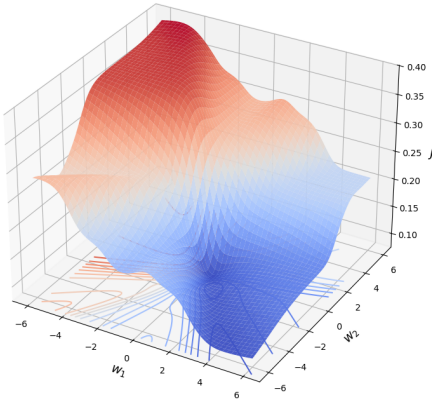
$$\mathbf{z} = \mathbf{X}\mathbf{w}, \qquad \texttt{// linear regression}$$
$$\sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}} \qquad \texttt{// maps z to probability-like (0,1)}$$

At $\mathbf{z} = \mathbf{0}$, $\sigma(\mathbf{z}) = 0.5$. So, with threshold 0.5, $\mathbf{z} = \mathbf{0}$ is the decision boundary. As linear regression doesn't have to be straight line, logistic regression's decision boundary does not have to be a straight line.

Linear boundary **Circular boundary**



The squared error loss function $L(\mathbf{w}, y_i) = \left(\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i\right)^2$, makes the cost function non-convex.

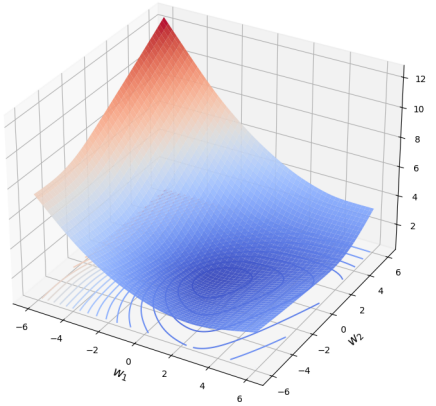


Negative log likelihood loss:

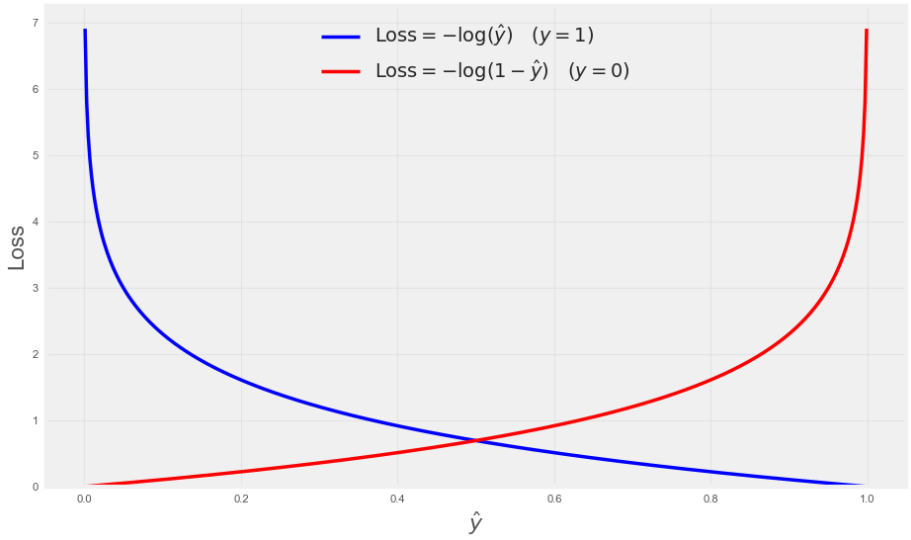
$$\hat{y}_i = \sigma\left(\mathbf{w}^T \mathbf{x}_i\right) \in (0, 1)$$

$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} \begin{cases} -\log(\hat{y}_i), & \text{if } y_i = 1, \\ -\log(1 - \hat{y}_i), & \text{if } y_i = 0 \end{cases}$$

The negative log likelihood loss function with $\sigma(z)$ makes the cost function convex.



Negative log likelihood also incentivizes appropriately: big mismatch $\implies$ big loss, small mismatch $\implies$ small loss.



$$z^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)}$$

Loss per example, $L\left(\mathbf{x}^{(i)}, \mathbf{w}\right) = -y^{(i)} \log\left(\sigma\left(z^{(i)}\right)\right) - \left(1 - y^{(i)}\right) \log\left(1 - \sigma\left(z^{(i)}\right)\right)$

$$\frac{\partial \sigma\left(z^{(i)}\right)}{\partial \mathbf{w}} = \frac{\partial \sigma(z^{(i)})}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \mathbf{w}} = \sigma(z^{(i)})(1 - \sigma(z^{(i)}))\mathbf{x}^{(i)}$$

$$\frac{\partial L\left(\mathbf{x}^{(i)}, \mathbf{w}\right)}{\partial \mathbf{w}} = \left(\sigma\left(z^{(i)}\right) - y^{(i)}\right) \mathbf{x}^{(i)}$$

$$\frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial L\left(\mathbf{x}^{(i)}, \mathbf{w}\right)}{\partial \mathbf{w}}$$

More concisely,

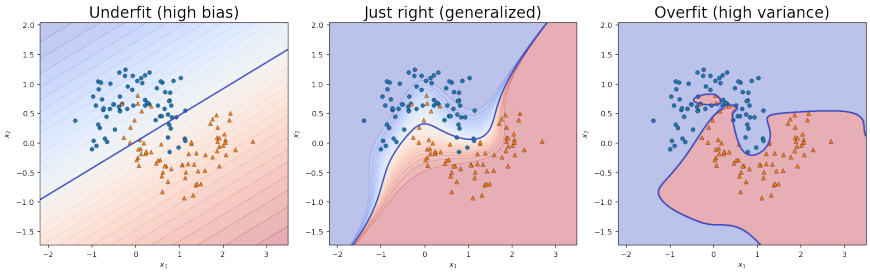
$$\mathbf{z} = \mathbf{X}\mathbf{w}$$

$$J(\mathbf{X}, \mathbf{w}) = -\frac{1}{m}\left[\mathbf{y}^T \log(\sigma(\mathbf{z})) + (\mathbf{1} - \mathbf{y})^T \log(1 - \sigma(\mathbf{z}))\right]$$

$$\frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}} = \frac{1}{m}\mathbf{X}^T(\sigma(\mathbf{z}) - \mathbf{y})$$

Gradient descent: $\mathbf{w} = \mathbf{w} - \alpha \frac{\partial J(\mathbf{X}, \mathbf{w})}{\partial \mathbf{w}}$

2 Generalization



2.1 Address overfit

1. Collect more training examples keeping the model as it is.
2. Select a smaller set of informative features to simplify the model. Regularizarion disables features in a gradual manner.

2.1.1 Regularization

L2:

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w}) = \frac{1}{m}\|\text{error}(\mathbf{w})\|^2$$

$$\texttt{gradient descent:}\quad \mathbf{w} = \mathbf{w} - \alpha\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$$

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J_{\text{reg}}(\mathbf{w}) = \frac{1}{m}\left(\|\text{error}(\mathbf{w})\|^2 + \lambda\|\mathbf{w}\|^2\right)$$

$$\texttt{gradient descent (reg):}\quad \mathbf{w} = \underbrace{\left(1 - \alpha\cdot\frac{2\lambda}{m}\right)}_{\text{shrinkage}}\mathbf{w} - \alpha\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \qquad \texttt{//}\ (\alpha\cdot{}^{2\lambda}/_{m}) \sim 0$$

Here, $\lambda\|\mathbf{w}\|^2$ encourages smaller parameter values. With bigger λ's, the parameters approach zero.

We can think regularization in terms of a constrained optimization problem:

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w})$$

$$\texttt{with constraint:}\quad \|\mathbf{w}\|^2 \leqslant r^2$$

In other words, minimize $J(\mathbf{w})$ but do not let $\mathbf{w}$ go beyond a ball of radius r.

L1:

$$\underset{\mathbf{w}}{\texttt{minimize}}\ J_{\text{reg}}(\mathbf{w}) = \frac{1}{m}\left(\|\text{error}(\mathbf{w})\|^2 + \lambda\|\mathbf{w}\|_1\right)$$

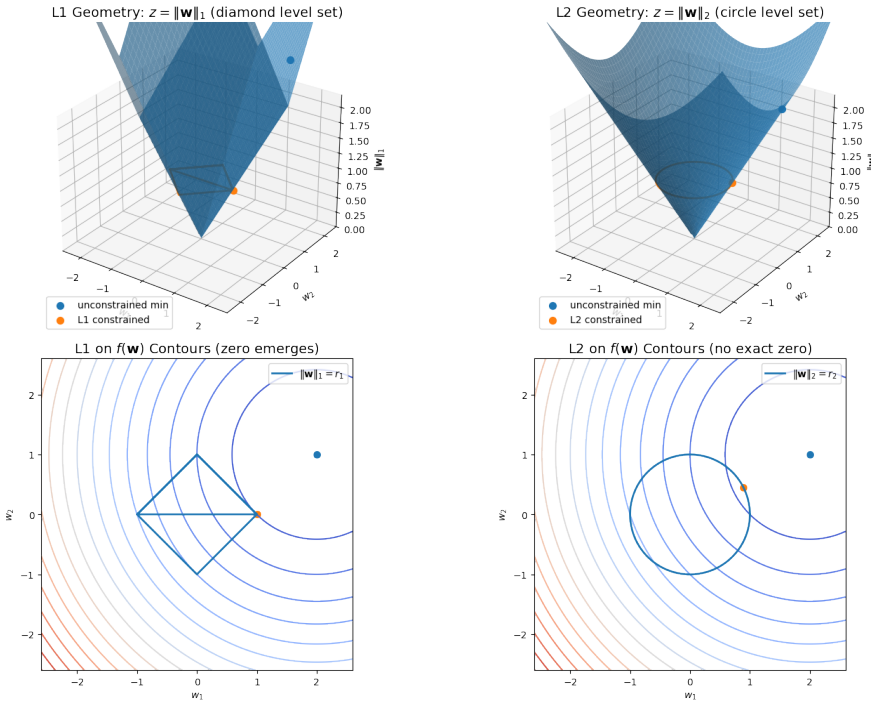
$$\|\mathbf{w}\|_1 = \sum_i |w_i|$$

$$\texttt{gradient descent (reg):}\quad \mathbf{w} = \mathbf{w} - \underbrace{\alpha \cdot \frac{\lambda}{m}\,\text{sign}(\mathbf{w})}_{\substack{\text{constant pull}\\ \text{towards 0}}} - \alpha \cdot \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$$

The corresponding constrained optimization:

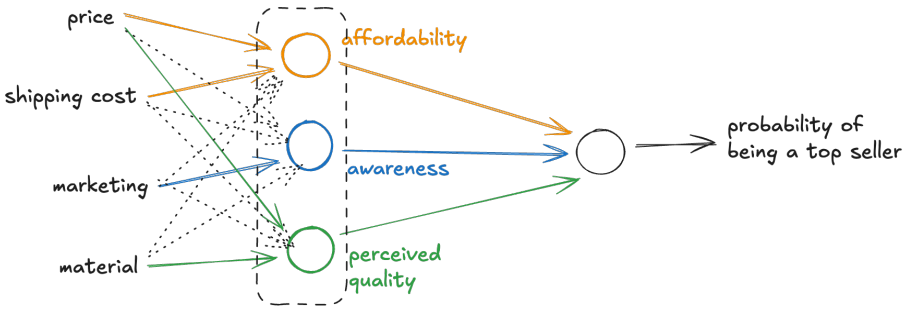
$$\underset{\mathbf{w}}{\texttt{minimize}}\ J(\mathbf{w})$$

$$\texttt{with constraint:}\quad \|\mathbf{w}\|_1 \leqslant r$$



With L1 regularization, parameters for less important features can be exactly zero.

3 Neural Networks



- A neural network realizes a piecewise differentiable function.
- In multi-layer perceptron (MLP), an individual neuron can do linear or logistic regression.
- Intermediate layers of neurons pick out features in a hierarchical manner. Left to right, the features become more abstract – feature from inputs, feature from features.
- Unlike other ML algorithms, a neural network's performance scales well with data.

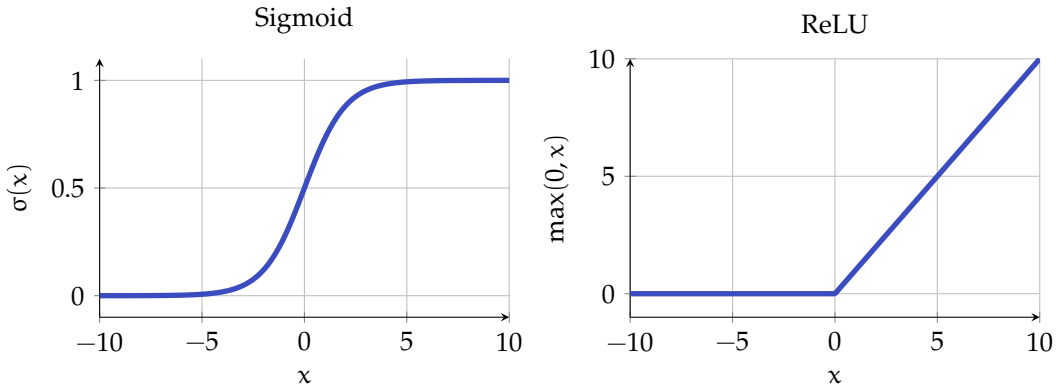
```
layer_1 = keras.layers.Dense(
    units=3,
    activation="sigmoid"
)
layer_2 = keras.layers.Dense(
    units=1,
    activation="sigmoid"
)
model = keras.Sequential([layer_1, layer_2])
model.compile(optimizer="adam", loss="mse")
model.fit(X, y, epochs=100)
model.predict(X_new)
```

3.1 Loss function

Target of optimization. For binary classification, we can use "binary cross entropy". For regression, we can use "mean squared error".

3.2 Activation function

Without activation function, a neural network can only implement a linear function. For example, it cannot implement the `XOR` function.



- Sigmoid$(x) \in (0,1)$ vs. ReLU$(x) \in [0,\infty)$. So, ReLU(x) allows large positive outputs.
- Output layer:
  - Binary classification: Sigmoid
  - Non-negative Regression: ReLU
  - Regression: None.
- Hidden layer: ReLU. Because:
  - Faster to compute
  - Flat regions have near-zero gradients, so gradient descent takes tiny steps and progresses slowly there. Sigmoid has two flat regions, ReLU has one.
- The off feature (0-region) of ReLU enables a neural network to stitch together linear segments to model a complex non-linear function.

3.2.1 Softmax

For multi-class classification. For n classes, activation of one of the n output neurons:

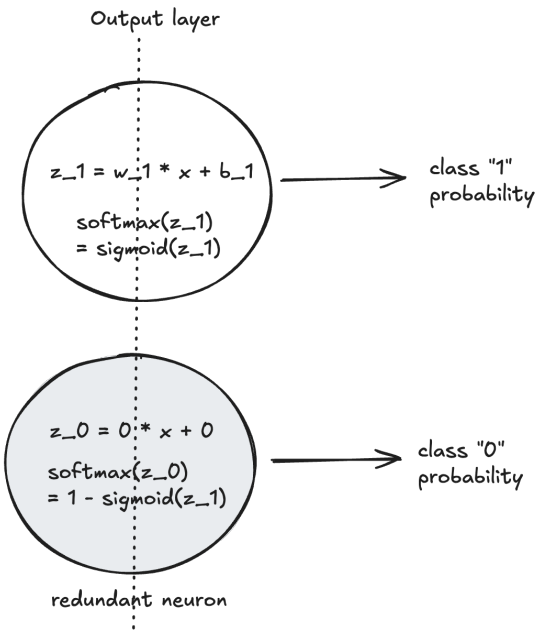
$$z_j = \mathbf{w}_j \cdot \mathbf{x} + b_j,\ j = 1, 2, \ldots, n \quad \texttt{// logits}$$
$$a_j = \Pr(y = j \mid \mathbf{x}) = \text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{n} e^{z_k}}$$

- $\text{Softmax}(z + c) = \text{Softmax}(z)$
- When $n = 2$, Softmax $\equiv$ Sigmoid:

$$a_1 = \text{Softmax}(z_1) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} = \frac{1}{1 + e^{-(z_1 - z_0)}}$$

When $z = 0$, $a_1 = \text{Sigmoid}(z_1), a_0 = 1 - a_1$

As if we have two output neurons, one with all parameters set to 0.



Training logits and applying Softmax at inference time improves numerical stability:

```
model = Sequential(
    [
        Dense(units=25, activation="relu"),
        Dense(units=15, activation="relu"),
        # outputs logits
        Dense(units=10, activation="linear")
    ]
)
model.compile(
    optimizer="adam",
    # train logits (z's)
    loss=SparseCategoricalCrossentropy(from_logits=True)
)
mode.fit(X, y, epochs=100)
logits = model(X_new)
probabilities = tf.nn.softmax(logits)
```

3.3 Optimizer

3.3.1 Gradient descent

$$\mathbf{w} = \mathbf{w} - \alpha \, \nabla J(\mathbf{w})$$

Learning rate α is fixed.

3.3.2 Adam (ADAptive Moment estimation)

$$\mathbf{w} = \mathbf{w} - \mathbf{A} \, \nabla J(\mathbf{w}), \text{ with } \mathbf{A} = \text{diag}(\alpha_1, \alpha_2, \ldots, \alpha_n)$$

- Generally faster convergence than gradient descent.
- Each parameter has its own learning rate.
- Each learning rate adapts on the fly based on earlier steps.

3.4 Layers

3.4.1 Dense layer

Activation of a neuron is a function of the activations of *all neurons* in the previous layer.

3.4.2 Convolutional layer

Activation of a neuron is a function of the activations of *a limited number of neurons* (overlapping sliding window) in the previous layer.

- Faster computation
- Less prone to overfitting

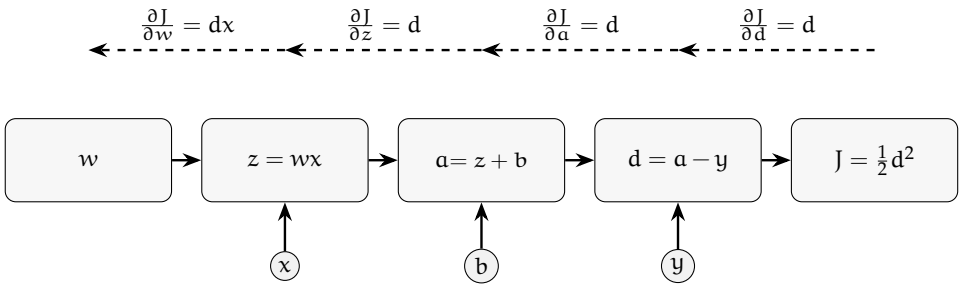
3.5 Backpropagation

```
J, w = sympy.symbols("J, w")
J = 1 / (1 + sympy.exp(-w))
# symbolic differentiation
dJ_dw = sympy.diff(J, w)
dJ_dw.subs([(w, 2)])
```

3.5.1 Computation graph

Evaluate DAG in topological order in forward-prop and in topological order of the reversed DAG in backprop.

$$
\begin{aligned}
J &= \frac{1}{2}d^2 \\
d &= a - y \\
a &= z + b \\
z &= wx
\end{aligned}
$$



With N nodes and P parameters, $T(\text{backprop}) = \mathcal{O}(N + P)$

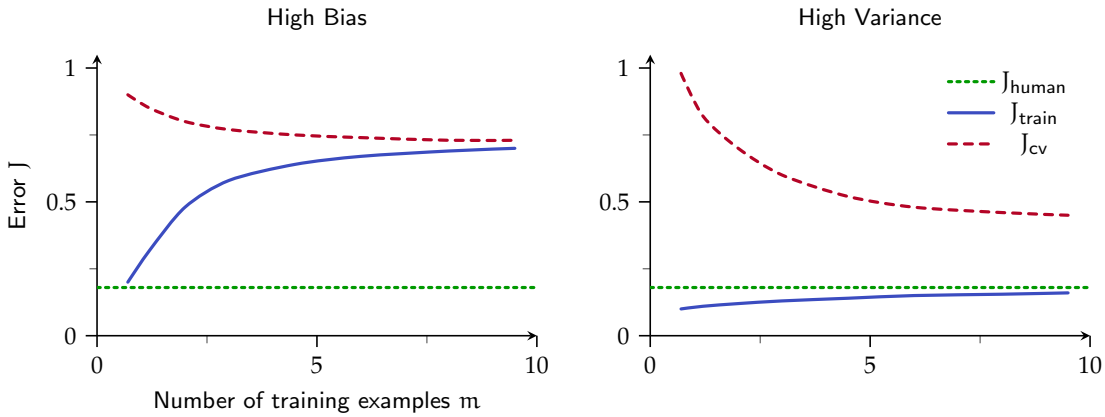
3.5.2 Autodiff

Tensorflow (or PyTorch) takes care of backprop for us via automatic differentiation.

4 Bias and variance

- High bias: J_{train} is large. $J_{train} \approx J_{cv}$. More data alone isn't going to help.
- High variance: J_{train} is small. $J_{cv} \gg J_{train}$. More data along can help.
- High bias and high variance: J_{train} is large. $J_{cv} \gg J_{train}$. For part of the input high bias, for some other part of the input high variance. Can happen with neural networks.
- With regularization: too big λ causes high bias, too small λ causes high variance.
- Large $(J_{train} - J_{human}) \implies$ high bias. Large $(J_{train} - J_{cv} \implies$ high variance.

4.1 Learning curve



5 References

- Machine learning specialization – Coursera